

Plan de Migración de Software

Sistema AFIP Mobile – Gestión Financiera Personal

Versión: 1.0.0

Fecha: Junio 2026

Clasificación: Documento Técnico Interno

1. Introducción

1.1 Propósito

Este documento define el plan para migrar el sistema **AFIP Mobile** desde el entorno de desarrollo local hacia un entorno de producción, garantizando la disponibilidad, integridad de los datos y la continuidad del servicio durante y después del proceso.

1.2 Alcance

La migración comprende:

- Migración de la base de datos MySQL desde localhost a servidor de producción
- Despliegue de la API Node.js en servidor cloud/VPS
- Publicación del frontend web en servidor/CDN
- Publicación de la aplicación mobile en Google Play Store (Android) y/o App Store (iOS)

1.3 Equipo de Migración

Rol	Responsabilidad
Líder de Migración	Coordinación general, aprobaciones
DBA	Migración y validación de la base de datos
DevOps / Backend	Despliegue de la API y frontend web
Desarrollador Mobile	Publicación de la app en stores

QA	Validación funcional post-migración
----	-------------------------------------

2. Inventario del Entorno Actual (Desarrollo)

2.1 Componentes a Migrar

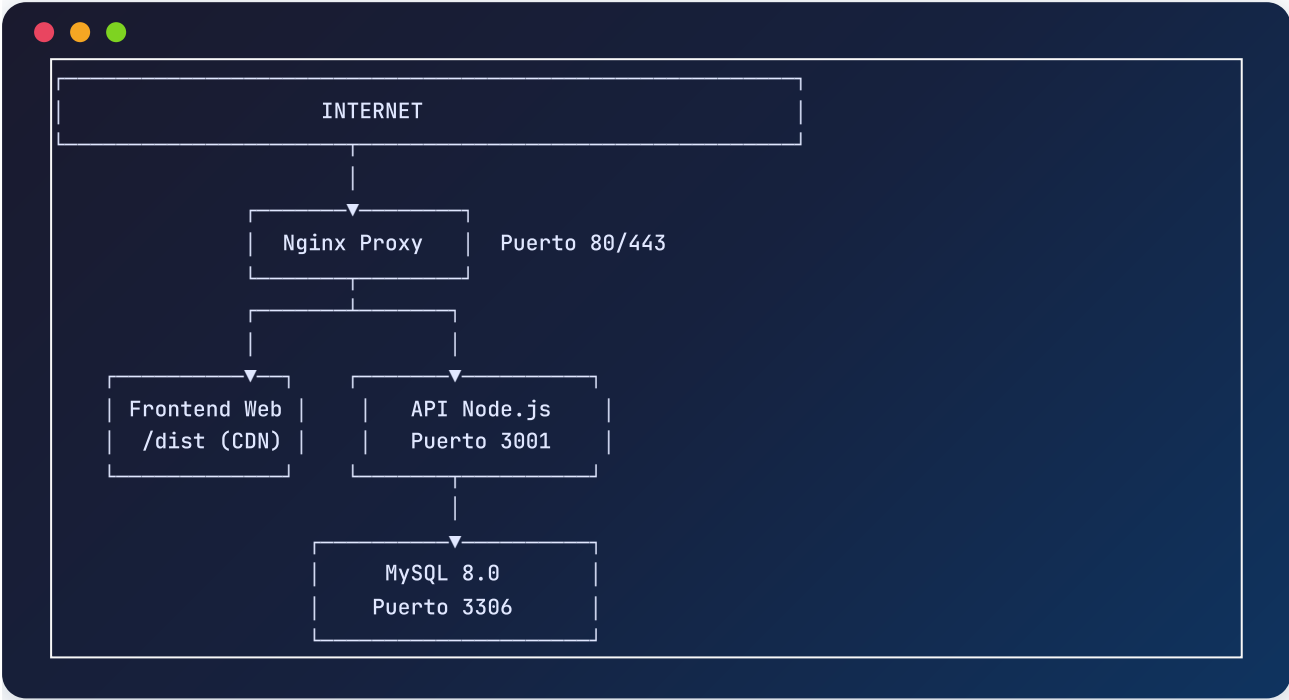
Componente	Tecnología	Versión	Ubicación Actual
Base de Datos	MySQL 8.0	1.0	localhost:3306
API REST	Node.js + Express 5	1.0.0	localhost:3001
Frontend Web	React 18 + Vite + Tailwind	1.0	localhost:5173
App Mobile	Flutter / Dart	1.0.0+1	Emulador / Dispositivo local

2.2 Base de Datos – Tablas a Migrar

Tabla	Descripción	Opciones de Migración
users	Usuarios del sistema	Completa (con claves UUID)
accounts	Cuentas financieras	Completa
categories	Categorías (con predefinidas)	Completa + seed de producción
budgets	Presupuestos	Completa
expenses	Gastos registrados	Completa
incomes	Ingresos registrados	Completa
income_assignments	Asignaciones de rubros	Completa
goals	Metas financieras	Completa
debts	Deudas	Completa
debtPayments	Pagos de deudas	Completa

3. Entorno de Producción Objetivo

3.1 Infraestructura Recomendada



3.2 Especificaciones del Servidor de Producción

Componente	Especificación
Proveedor sugerido	AWS EC2, DigitalOcean, Hostinger VPS
Sistema Operativo	Ubuntu Server 22.04 LTS
CPU	2 vCPU
RAM	4 GB
Almacenamiento	50 GB SSD
Transferencia	2 TB/mes

4. Fases de Migración

Fase 0 – Preparación (2-3 días antes)

4.0.1 Verificar lista de pre-migración

- ☐ El repositorio Git está actualizado con la versión final
- ☐ Todos los tests pasan en entorno de desarrollo
- ☐ La documentación técnica está completa
- ☐ El servidor de producción está provisionado y accesible
- ☐ Se ha tomado un backup completo del entorno de desarrollo

4.0.2 Provisionar servidor de producción

```
# Conectar al servidor
ssh usuario@IP_SERVIDOR_PRODUCCION

# Actualizar sistema
sudo apt update && sudo apt upgrade -y

# Instalar dependencias base
sudo apt install -y curl git nginx mysql-server ufw

# Configurar firewall
sudo ufw allow ssh
sudo ufw allow 80
sudo ufw allow 443
sudo ufw enable

# Instalar Node.js 18
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
```

Fase 1 – Migración de la Base de Datos

Paso 1.1 – Exportar desde desarrollo

```
# En el entorno de DESARROLLO
mysqldump -u root -p node_project_api \
  --single-transaction \
  --routines \
  --triggers \
  > afip_mobile_migration_$(date +%Y%m%d).sql
```

```
# Verificar el dump
wc -l afip_mobile_migration_$(date +%Y%m%d).sql
```

Paso 1.2 – Transferir al servidor de producción

```
scp afip_mobile_migration_YYYYMMDD.sql \
  usuario@IP_PRODUCCION:/tmp/
```

Paso 1.3 – Importar en producción

```
# En el servidor de PRODUCCIÓN
sudo mysql -u root -p << EOF
CREATE DATABASE IF NOT EXISTS node_project_api
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER IF NOT EXISTS 'afip_user'@'localhost' IDENTIFIED BY 'PASSWORD_SEGURA';
GRANT ALL PRIVILEGES ON node_project_api.* TO 'afip_user'@'localhost';
FLUSH PRIVILEGES;
EOF

# Importar datos
mysql -u afip_user -p node_project_api < /tmp/afip_mobile_migration_YYYYMMDD.sql
```

Paso 1.4 – Verificar migración de DB

```
mysql -u afip_user -p node_project_api << EOF
SHOW TABLES;
SELECT COUNT(*) as total_users FROM users;
SELECT COUNT(*) as total_accounts FROM accounts;
SELECT COUNT(*) as total_expenses FROM expenses;
EOF
```

Fase 2 – Migración de la API Node.js

Paso 2.1 – Clonar código en el servidor

```
# En el servidor de PRODUCCIÓN
cd /var/www
sudo git clone https://github.com/tu-usuario/afip-mobile.git
cd afip-mobile/api/Node_api
```

Paso 2.2 – Configurar variables de entorno de producción

```
sudo nano /var/www/afip-mobile/api/Node_api/.env
```

```
SERVER_PORT=3001
DB_NAME=node_project_api
DB_HOST=127.0.0.1
DB_USER=afip_user
DB_PASSWORD=PASSWORD_SEGURA
DB_PORT=3306
DB_DIALECT=mysql
JWT_SECRET=CLAVE_JWT_PRODUCCION_SEGURA_64_CHARS
```

Paso 2.3 – Instalar dependencias y configurar sync

```
npm install --production

# En src/index.js: cambiar modelsApp(true) a modelsApp(false)
```

Paso 2.4 – Instalar y configurar PM2 (gestor de procesos)

```
sudo npm install -g pm2

# Iniciar la API con PM2
pm2 start src/index.js --name "afip-api" --interpreter node

# Configurar inicio automático
```

```
pm2 startup
pm2 save
```

Paso 2.5 – Verificar API en producción

```
pm2 status
curl http://localhost:3001/api/v1/user
# Esperado: JSON con error 401 (no autenticado)
```

Fase 3 – Migración del Frontend Web

Paso 3.1 – Build de producción

```
# En el entorno de DESARROLLO
cd front-end-web/finance-frontend-react
echo "VITE_API_URL=https://tu-dominio.com/api/v1" > .env.production
npm run build
# Se genera la carpeta /dist
```

Paso 3.2 – Transferir build al servidor

```
scp -r dist/ usuario@IP_PRODUCCION:/var/www/afip-web/
```

Paso 3.3 – Configurar Nginx

```
# /etc/nginx/sites-available/afip-mobile
server {
    listen 80;
    server_name tu-dominio.com www.tu-dominio.com;
    root /var/www/afip-web;
    index index.html;

    # Frontend SPA
```

```
location / {
    try_files $uri $uri/ /index.html;
    add_header Cache-Control "no-cache";
}

# Proxy a la API
location /api {
    proxy_pass http://localhost:3001;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
}
```

```
sudo ln -s /etc/nginx/sites-available/afip-mobile /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

Paso 3.4 – Configurar HTTPS (Let's Encrypt)

```
sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx -d tu-dominio.com
```

Fase 4 – Migración de la Aplicación Mobile

Paso 4.1 – Actualizar URL de la API en código Flutter

```
// lib/services/api_service.dart (o equivalente)
static const String baseUrl = 'https://tu-dominio.com/api/v1';
```

Paso 4.2 – Compilar release para Android

```
cd front-end-mobile/application_login
```



```
# Compilar APK de release
flutter build apk --release

# O compilar App Bundle (recomendado para Play Store)
flutter build appbundle --release
```

Archivo generado: `build/app/outputs/bundle/release/app-release.aab`

Paso 4.3 – Publicar en Google Play Store

1. Acceder a [Google Play Console](#)
2. Crear nueva aplicación o actualizar versión existente
3. Subir el archivo `.aab`
4. Completar metadatos, capturas de pantalla y descripción
5. Enviar a revisión

5. Validaciones Post-Migración

5.1 Checklist de Validación

Nº	Validación	Método	Resultado Esperado
1	API responde en producción	<code>curl https://tu-dominio.com/api/v1/user</code>	HTTP 401
2	Registro de nuevo usuario	POST <code>/api/v1/user</code>	HTTP 200 + token JWT
3	Login de usuario existente	POST <code>/api/v1/login</code>	HTTP 200 + token
4	Acceso a datos protegidos	GET <code>/api/v1/cuentas</code> con token	HTTP 200 + datos
5	Frontend web carga correctamente	Navegar a <code>https://tu-dominio.com</code>	Pantalla de login
6	App mobile conecta a API	Intentar login desde la app	Ingreso exitoso
7	HTTPS funcional	Navegar con HTTPS	Certificado válido

6. Plan de Rollback

Si durante la migración ocurre algún fallo crítico, se ejecuta el siguiente plan de rollback:

Criterios para activar el Rollback

- La API no responde después de 30 minutos de diagnóstico
- Pérdida de datos detectada en la base de datos
- Más del 30% de las validaciones post-migración fallan
- El tiempo de migración supera el doble del estimado

Procedimiento de Rollback

```
# Paso 1: Detener la API en producción
pm2 stop afip-api

# Paso 2: Restaurar base de datos desde backup pre-migración
mysql -u root -p node_project_api < /backups/pre_migration_backup.sql

# Paso 3: Restaurar código anterior (si se usó deploy de rama)
cd /var/www/afip-mobile
git checkout main-stable # Rama estable anterior

# Paso 4: Verificar y reiniciar API
npm install
pm2 restart afip-api

# Paso 5: Restaurar la versión web anterior
cp -r /backups/web_dist_backup/dist /var/www/afip-web/

# Paso 6: Validar restauración
curl https://tu-dominio.com/api/v1/user
```

7. Riesgos y Mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Pérdida de datos durante migración DB	Baja	Alto	Backup completo previo + checksum
API no conecta a MySQL en producción	Media	Alto	Verificar firewall y credenciales
Errores de CORS en producción	Media	Medio	Configurar headers CORS en Nginx/Express
Tiempo de inactividad > RTO	Baja	Alto	Ejecución en horario de bajo tráfico
Credenciales JWT inválidas	Baja	Medio	Uso de secretos de producción verificados

8. Cronograma de Migración

Fase	Actividad	Duración Estimada
Fase 0	Preparación y provisionamiento	1-2 días
Fase 1	Migración de base de datos	2-4 horas
Fase 2	Despliegue de API	2-3 horas
Fase 3	Despliegue de frontend web	1-2 horas
Fase 4	Publicación de app mobile	1-3 días (revisión de store)
Validación	QA post-migración	4-8 horas
Total		~3-5 días hábiles